Hindley-Milner Type Inference

Concepts of Programming Languages

CAS CS 320

Outline

- Discuss **Hindley–Milner type inference**
- Look at rules for **constraint-based inference**
- Walk through some **examples**

Motivation

Explicit Typing

```
let add (x : int) (y : int) : int = x + y
let k (x : int) (y : bool) : int = x
let _ : unit = assert(add 2 3 = k 5 false)
```

Explicit Typing

```
let add (x : int) (y : int) : int = x + y
let k (x : int) (y : bool) : int = x
let _ : unit = assert(add 2 3 = k 5 false)
```

We've implemented PLs where function arguments *must* be type-annotated

Explicit Typing

```
let add (x : int) (y : int) : int = x + y
let k (x : int) (y : bool) : int = x
let _ : unit = assert(add 2 3 = k 5 false)
```

We've implemented PLs where function arguments *must* be type-annotated

This is closer to what's done in PLs like **Java**

Implicit Types

```
let add x y = x + y
let k x y = x
let _ = assert(add 2 3 = k 5 false)
```

Implicit Types

```
let add x y = x + y
let k x y = x
let _ = assert(add 2 3 = k 5 false)
```

Type inference is the process of determining what type-annotations we *could* have used

Implicit Types

```
let add x y = x + y
let k x y = x
let _ = assert(add 2 3 = k 5 false)
```

Type inference is the process of determining what type-annotations we *could* have used

What type should we give k?

Recall: Parametric Polymorphism

```
let rec rev = function
  | [] -> []
  | x :: xs -> rev xs @ [x]
```

Recall: Parametric Polymorphism

```
let rec rev = function
  | [] -> []
  | x :: xs -> rev xs @ [x]
```

Parametric polymorphism allows for functions which are agnostic to the types of its inputs

Recall: Parametric Polymorphism

```
let rec rev = function
  | [] -> []
  | x :: xs -> rev xs @ [x]
```

Parametric polymorphism allows for functions which are agnostic to the types of its inputs

For example, we can write a single reverse function and use it in multiple contexts

Recall: Type Uniqueness

Recall: Type Uniqueness

Lemma. If $\Gamma \vdash e : \tau_1$ and $\Gamma \vdash e : \tau_2$ then $\tau_1 = \tau_2$

Recall: Type Uniqueness

> **Lemma.** If $\Gamma \vdash e : \tau_1$ and $\Gamma \vdash e : \tau_2$ then $\tau_1 = \tau_2$

Our lives are a lot easier if we can give every expression a unique type

Recall: Type Uniqueness

> **Lemma.** If $\Gamma \vdash e : \tau_1$ and $\Gamma \vdash e : \tau_2$ then $\tau_1 = \tau_2$

Our lives are a lot easier if we can give every expression a unique type

We'd like this to hold, even with polymorphism

Recall: Type Variables

```
let id : 'a -> 'a = fun x -> x
```

Recall: Type Variables

```
let id : 'a -> 'a = fun x -> x
```

The "parametric" part is that types have *variables*

Recall: Type Variables

```
let id : 'a -> 'a = fun x -> x
```

The "parametric" part is that types have *variables*

Type variables are *instantiated* at particular types

Recall: Type Variables

```
let id : 'a -> 'a = fun x -> x
```

The "parametric" part is that types have *variables*

Type variables are *instantiated* at particular types

They are similar to expression variables, e.g., we need to define *type-level substitution*

Recall: Quantification

```
let id : 'a . 'a -> 'a = fun x -> x
```

Recall: Quantification

```
let id : 'a . 'a -> 'a = fun x -> x
```

Types variables in OCaml are actually **quantified**

Recall: Quantification

```
let id : 'a . 'a -> 'a = fun x -> x
```

Types variables in OCaml are actually **quantified**

Just like with expression variables, we don't like *unbound* type variables

Recall: Quantification

$\forall$ 'a. 'a → 'a

```
let id : 'a . 'a -> 'a = fun x -> x
```

Types variables in OCaml are actually **quantified**

Just like with expression variables, we don't like *unbound* type variables

We read this "**id** has type **t -> t** for any type **t**"

Recall: Quantification

(unique) polymorphic type

```
let id : 'a . 'a -> 'a = fun x -> x
```

Types variables in OCaml are actually **quantified**

Just like with expression variables, we don't like *unbound* type variables

We read this "**id** has type **t -> t** for any type **t**"

Aside: System F

```
let id_int : int -> int = fun (x : int) -> x
let id : 'a . 'a -> 'a = fun 'a -> fun (x : 'a) -> x

let test1 = id_int 2
let test2 = id int 2
let test3 = id string "two"
```

Aside: System F

```
let id_int : int -> int = fun (x : int) -> x
let id : 'a . 'a -> 'a = fun 'a -> fun (x : 'a) -> x

let test1 = id_int 2
let test2 = id int 2
let test3 = id string "two"
```

System F (**second-order lambda calculus**) was introduced by Jean-Yves Girard and John C. Reynolds in the 1970s

Aside: System F

```
let id_int : int -> int = fun (x : int) -> x
let id : 'a . 'a -> 'a = fun 'a -> fun (x : 'a) -> x

let test1 = id_int 2
let test2 = id int 2
let test3 = id string "two"
```

System F (**second-order lambda calculus**) was introduced by Jean-Yves Girard and John C. Reynolds in the 1970s

The basic idea: Introduce types into the language itself so we can *pass them as arguments to functions*

Aside: System F

```
let id_int : int -> int = fun (x : int) -> x
let id : 'a . 'a -> 'a = fun 'a -> fun (x : 'a) -> x

let test1 = id_int 2
let test2 = id int 2
let test3 = id string "two"
```

System F (**second-order lambda calculus**) was introduced by Jean-Yves Girard and John C. Reynolds in the 1970s

The basic idea: Introduce types into the language itself so we can *pass them as arguments to functions*

The big problem: without type annotations type checking is undecidable

Interlude:
Compact Derivations

The Problem

Derivations take up a lot of horizontal space

We've been careful (sort of) to choose expressions with short derivations in lecture

We won't be able to do this moving forward

The Problem

$$\dfrac{\dfrac{}{\{\} \vdash \texttt{2} : \texttt{int}}\,(\text{intLit}) \qquad \dfrac{\dfrac{}{\{\texttt{y} : \texttt{int}\} \vdash \texttt{y} : \texttt{int}}\,(\text{var}) \qquad \dfrac{}{\{\texttt{y} : \texttt{int}\} \vdash \texttt{y} : \texttt{int}}\,(\text{var})}{\{\texttt{y} : \texttt{int}\} \vdash \texttt{y} + \texttt{y} : \texttt{int}}\,(\text{intAdd})}{\{\} \vdash \texttt{let y = 2 in y} + \texttt{y} : \texttt{int}}\,(\text{let})$$

Derivations take up a lot of horizontal space

We've been careful (sort of) to choose expressions with short derivations in lecture

We won't be able to do this moving forward

Visualizing Trees

```
.
├── bin
│   ├── dune
│   └── main.ml
├── dune-project
├── interp2.opam
├── lib
│   ├── dune
│   ├── interp2.ml
│   ├── lexer.mll
│   ├── parser.mly
│   └── utils.ml
├── spec.pdf
└── test
    ├── dune
    └── test_interp2.ml
```

Finding a "good" visualization of trees is an art

Moving forward we'll use the *file-tree format* for writing derivations (this is what is done in OCP)

It's more horizontally space-efficient

Example

Example

$$\dfrac{\dfrac{}{\{\} \vdash \texttt{2} : \texttt{int}}\,(\text{intLit}) \qquad \dfrac{\dfrac{}{\{\texttt{y} : \texttt{int}\} \vdash \texttt{y} : \texttt{int}}\,(\text{var}) \qquad \dfrac{}{\{\texttt{y} : \texttt{int}\} \vdash \texttt{y} : \texttt{int}}\,(\text{var})}{\{\texttt{y} : \texttt{int}\} \vdash \texttt{y} + \texttt{y} : \texttt{int}}\,(\text{intAdd})}{\{\} \vdash \texttt{let y = 2 in y} + \texttt{y} : \texttt{int}}\,(\text{let})$$

$\{\} \vdash \texttt{let y = 2 in y} + \texttt{y} : \texttt{int}$ (let)

- $\{\} \vdash \texttt{2} : \texttt{int}$ (intLit)
- $\{\texttt{y} : \texttt{int}\} \vdash \texttt{y} + \texttt{y} : \texttt{int}$ (addInt)
  - $\{\texttt{y} : \texttt{int}\} \vdash \texttt{y} : \texttt{int}$ (var)
  - $\{\texttt{y} : \texttt{int}\} \vdash \texttt{y} : \texttt{int}$ (var)

Practice Problem

- $\cdot \vdash$ `fun f -> fun x -> f (x + 1) : (int -> int) -> int -> int`

Give a typing derivation in compact form of the above judgment using 320Caml typing rules

Answer

- $\vdash$ `fun f -> fun x -> f (x + 1) : (int -> int) -> int -> int`

Hindley-Milner

High Level

High Level

Hindley–Milner type systems are typed λ–calculi with parametric polymorphism

High Level

Hindley–Milner type systems are typed λ–calculi with parametric polymorphism

Use in many functional PLs (OCaml, Haskell, Elm)

High Level

Hindley–Milner type systems are typed λ–calculi with parametric polymorphism

Use in many functional PLs (OCaml, Haskell, Elm)

They allow for *restricted* type quantification

High Level

Hindley–Milner type systems are typed λ–calculi with parametric polymorphism

Use in many functional PLs (OCaml, Haskell, Elm)

They allow for *restricted* type quantification

Inference is decidable and (somewhat) efficient

Type Inference (High Level)

$$\Gamma \vdash e : \tau \dashv \mathscr{C}$$

Type Inference (High Level)

$$\Gamma \vdash e : \tau \dashv \mathscr{C}$$

The constraint-based type inference process:

Type Inference (High Level)

$$\Gamma \vdash e : \tau \dashv \mathscr{C}$$

The constraint-based type inference process:

1. Derive $\Gamma \vdash e : \tau$ *relative to some constraints* $\mathscr{C}$

Type Inference (High Level)

$$\Gamma \vdash e : \tau \dashv \mathscr{C}$$

The constraint-based type inference process:

1. Derive $\Gamma \vdash e : \tau$ *relative to some constraints* $\mathscr{C}$
2. Use the $\mathscr{C}$ to determine the "actual" type of e in Γ

Type Inference (High Level)

$$\Gamma \vdash e : \tau \dashv \mathscr{C}$$

The constraint-based type inference process:

today

1. Derive $\Gamma \vdash e : \tau$ *relative to some constraints* $\mathscr{C}$
2. Use the $\mathscr{C}$ to determine the "actual" type of e in Γ

Example (by Intuition)

```
fun f -> fun x -> f (x + 1)
```

$\emptyset \vdash \text{fun } f \to \text{fun } x \to f\ (x+1) : ?_1 \to ?_2 \to ?_3$

$\{f : ?_1\} \vdash \text{fun } x \to f\ (x+1) : ?_2 \to ?_3$

$\{f : ?_1,\ x : ?_2\} \vdash f\ (x+1) : ?_3$

$\{f : ?_1,\ x : ?_2\} \vdash f : ?_1$

$\{\ldots\} \vdash x + 1 : \text{int}$

$\{f : ?_1,\ x : ?_2\} \vdash x : ?_2$

$\{\ldots\} \vdash 1 : \text{int}$

e

$?_2 \doteq \boxed{\text{int}}$

$?_1 \doteq \boxed{\text{int} \to ?_3}$

$\tau : \boxed{?_1} \to \boxed{?_2} \to ?_3$

$\tau : (\text{int} \to ?_3) \to \text{int} \to ?_3$

Hindley-Milner Light (Syntax)

```
<expr>  ::=  fun <var> -> <expr> | <expr> <expr>
          |  let <var> = <expr> in <expr>
          |  if <expr> then <expr> else <expr>
          |  <expr> + <expr> | <expr> = <expr>
          |  <int> | <var>
<mty>   ::=  int | bool | <tyvar> | <mty> -> <mty>
<ty>    ::=  <tyvar>.<ty> | <mty>
```

type parameters : 'a, 'b, ...

type quantifiers

Hindley-Milner Light (Mathematical)

$$
\begin{aligned}
e ::= \; & \lambda x \,.\, e \mid ee \\
& \mid \text{let } x = e \text{ in } e \\
& \mid \text{if } e \text{ then } e \text{ else } e \\
& \mid e + e \mid e = e \\
& \mid n \mid x \\
\sigma ::= \; & \text{int} \mid \text{bool} \mid \alpha \mid \sigma \to \sigma \\
\tau ::= \; & \sigma \mid \forall \alpha \,.\, \tau
\end{aligned}
$$

Type Variables and Type Schemes

§

type parameters : $\alpha, \beta, \gamma, \eta, \varepsilon, \ldots$

$$\sigma ::= \text{int} \mid \text{bool} \mid \alpha \mid \sigma \to \sigma$$

$$\tau ::= \sigma \mid \forall \alpha . \tau$$

type quantifier

Type Variables and Type Schemes

$$\sigma ::= \text{int} \mid \text{bool} \mid \alpha \mid \sigma \to \sigma$$

$$\tau ::= \sigma \mid \forall \alpha . \tau$$

$\cancel{\forall \alpha . \alpha \to (\forall \beta . \beta \to \beta)}$

σ :

int

bool

bool $\to$ (int $\to$ bool)

$\alpha \to \beta \to$ int

τ

$\alpha \to \beta \to$ int

$\forall \gamma . \alpha \to \beta \to$ int

$\forall \alpha . \alpha \to \beta \to$ int

$\forall \alpha . \forall \beta . \alpha \to \beta \to$ int

Type Variables and Type Schemes

$$\sigma ::= \text{int} \mid \text{bool} \mid \alpha \mid \sigma \to \sigma$$

$$\tau ::= \sigma \mid \forall \alpha \,.\, \tau$$

σ represents **monotypes,** types with *no quantification*

Type Variables and Type Schemes

$$\sigma ::= \text{int} \mid \text{bool} \mid \alpha \mid \sigma \to \sigma$$

$$\tau ::= \sigma \mid \forall \alpha \,.\, \tau$$

σ represents **monotypes,** types with *no quantification*

A type is **monomorphic** if it is a monotype with no type variables

Type Variables and Type Schemes

$$\sigma ::= \text{int} \mid \text{bool} \mid \alpha \mid \sigma \to \sigma$$

$$\tau ::= \sigma \mid \forall \alpha \,.\, \tau$$

σ represents **monotypes,** types with *no quantification*

A type is **monomorphic** if it is a monotype with no type variables

τ represents **type schemes,** types with some quantifiers

Type Variables and Type Schemes

$$\sigma ::= \text{int} \mid \text{bool} \mid \alpha \mid \sigma \to \sigma$$

$$\tau ::= \sigma \mid \forall \alpha . \tau$$

int → α

σ represents **monotypes,** types with *no quantification*

int → int

A type is **monomorphic** if it is a monotype with no type variables

∀α. α → β

τ represents **type schemes,** types with some quantifiers

∀α. ∀β. α → β

We say a type is **polymorphic** if it is a *closed* type scheme

Free Variables (Monotypes)

$\sigma ::= int \mid bool \mid \alpha \mid \sigma \to \sigma$

$FV(int) = \emptyset$

$FV(bool) = \emptyset$

$FV(\alpha) = \{\alpha\}$

$FV(t_1 \to t_2) = FV(t_1) \cup FV(t_2)$

t

$\gamma \to ((\alpha \to int) \to (\alpha \to \beta)) \to (\gamma \to \varepsilon)$

$FV(t) = \{\gamma, \alpha, \beta, \varepsilon\}$

Substitution (Monotypes)

$$[t/\alpha]\,\mathrm{int} = \mathrm{int}$$

$$[t/\alpha]\,\mathrm{bool} = \mathrm{bool}$$

$$[t/\alpha]\,\beta = \begin{cases} t & \alpha = \beta \\ \beta & \text{o.w.} \end{cases}$$

$$[t/\alpha]\,(t_1 \to t_2) = [t/\alpha]\,t_1 \to [t/\alpha]\,t_2$$

$$[\alpha \to \mathrm{int}/\beta]\,(\beta \to \beta) = (\alpha \to \mathrm{int}) \to (\alpha \to \mathrm{int})$$

Constraint-Based Inference

$$\Gamma \vdash e : \tau \dashv \mathscr{C}$$

Constraint-Based Inference

$$\Gamma \vdash e : \tau \dashv \mathscr{C}$$

Our typing rules keep track of **constraints**

Constraint-Based Inference

$$\Gamma \vdash e : \tau \dashv \mathscr{C}$$

Our typing rules keep track of **constraints**

Constraints say what must hold so e is well-typed

Constraint-Based Inference

$$\Gamma \vdash e : \tau \dashv \mathscr{C}$$

Our typing rules keep track of **constraints**

Constraints say what must hold so e is well-typed

The idea: We're formalizing the process of "collecting" constraints, as in our example

What is a constraint?

$$\tau_1 \doteq \tau_2$$

A **type constraint** is a predicate on types

The only kind we will consider:

"τ_1 should be the same as τ_2"

Enforcing this constraint is called **unifying** τ_1 and τ_2

Constraint-Based Inference

$$\Gamma \vdash e : \tau \dashv \mathscr{C}$$

Constraint-Based Inference

$$\Gamma \vdash e : \tau \dashv \mathscr{C}$$

The idea: For each rule:

Constraint-Based Inference

$$\Gamma \vdash e : \tau \dashv \mathscr{C}$$

The idea: For each rule:

» What is the *most general* type τ we could give e?

Constraint-Based Inference

$$\Gamma \vdash e : \tau \dashv \mathscr{C}$$

The idea: For each rule:

- What is the *most general* type τ we could give e?
- What must be true of τ, i.e., what *constrains* τ?

Constraint-Based Inference

$$\Gamma \vdash e : \tau \dashv \mathscr{C}$$

The idea: For each rule:

- What is the *most general* type τ we could give e?
- What must be true of τ, i.e., what *constrains* τ?

If we don't know the type, *we create a fresh type variable*

Let's see some typing rules...

HM⁻ (Typing Literals)

$$\frac{}{\Gamma \vdash true : bool \dashv \emptyset}$$

HM⁻ (Typing Operators)

$$\frac{\Gamma \vdash e_1 : \tau_1 \dashv \mathcal{C}_1 \qquad \Gamma \vdash e : \tau_2 \dashv \mathcal{C}_2}{\Gamma \vdash e_1 + e_2 : \mathrm{int} \dashv \mathcal{C}_1, \mathcal{C}_2, \tau_1 \doteq \mathrm{int}, \tau_2 \doteq \mathrm{int}}$$

$$\frac{\Gamma \vdash e_1 : \tau_1 : \mathcal{C}_1 \qquad \Gamma \vdash e_2 : \tau_2 \dashv \mathcal{C}_2}{\Gamma \vdash e_1 = e_2 : \mathrm{bool} \dashv \mathcal{C}_1, \mathcal{C}_2, \tau_1 \doteq \tau_2}$$

HM⁻ (Typing If-Expressions)

$$\frac{\Gamma \vdash e_1 : \tau_1 \to \mathcal{C}_1 \qquad \Gamma \vdash e_2 : \tau_2 \to \mathcal{C}_2 \qquad \Gamma \vdash e_3 : \tau_3 \to \mathcal{C}_3}{\Gamma \vdash \text{if } e_1 \text{ then } e_2 \text{ else } e_3 : \quad \to \mathcal{C}_1, \mathcal{C}_2, \mathcal{C}_3}$$

$$\tau_1 \doteq \text{bool}$$

$$\tau_2 \doteq \tau_3$$

Example

$\{x : \alpha, y : \beta\} \vdash \text{if } x \text{ then } x \text{ else } y : \tau \dashv \mathscr{C}$

$\{x : \alpha, y : \beta\} \vdash \text{if } x \text{ then } x \text{ else } y : \alpha \dashv \alpha \doteq \text{bool}$

$\alpha \doteq \beta$

$\{\ldots\} \vdash x : \alpha \dashv \emptyset$ (var)

$\{\ldots\} \vdash x : \alpha \dashv \emptyset$ (var)

$\{\ldots\} \vdash y : \beta \dashv \emptyset$ (var)

HM⁻ (Typing Functions)

$$\frac{\alpha \text{ is fresh} \qquad \Gamma, x : \alpha \vdash e : \tau \dashv \mathcal{C}}{\Gamma \vdash \text{fun } x \to e : \alpha \to \tau \dashv \mathcal{C}}$$

HM⁻ (Typing Application)

$$\frac{\Gamma \vdash e_1 : \tau_1 \dashv C_1 \qquad \Gamma \vdash e_2 : \tau_2 \dashv C_2 \qquad \alpha \text{ is fresh}}{\Gamma \vdash e_1\ e_2 : \alpha \dashv C_1, C_2, \tau_1 \doteq \tau_2 \to \alpha}$$

HM⁻ (Typing Variables)

σ is monotype

$$\frac{(x : \sigma) \in \Gamma}{\Gamma \vdash x : \sigma \dashv \emptyset}$$

$\tau = \forall \alpha . \sigma$

$$\frac{(x : \forall \alpha . \sigma) \in \Gamma \qquad \beta \text{ is fresh}}{\Gamma \vdash x : [\beta / \alpha] \sigma}$$

HM⁻ (Typing Variables)

$$\frac{(x : \forall \alpha_1 . \forall \alpha_2 \ldots \forall \alpha_k . \tau) \in \Gamma \qquad \beta_1, \ldots, \beta_k \ \texttt{are fresh}}{\Gamma \vdash x : [\beta_1/\alpha_1] \ldots [\beta_k/\alpha_k]\tau \dashv \varnothing} \ \texttt{(var)}$$

HM⁻ (Typing Variables)

$$\frac{(x : \forall \alpha_1 . \forall \alpha_2 \ldots \forall \alpha_k . \tau) \in \Gamma \qquad \beta_1, \ldots, \beta_k \ \texttt{are fresh}}{\Gamma \vdash x : [\beta_1/\alpha_1] \ldots [\beta_k/\alpha_k]\tau \dashv \varnothing} \ \texttt{(var)}$$

If x is declared in Γ, then x can be given the type τ *with all free variables replaced by* ***fresh variables***

HM⁻ (Typing Variables)

$$\frac{(x : \forall \alpha_1 . \forall \alpha_2 \ldots \forall \alpha_k . \tau) \in \Gamma \qquad \beta_1, \ldots, \beta_k \ \texttt{are fresh}}{\Gamma \vdash x : [\beta_1/\alpha_1] \ldots [\beta_k/\alpha_k]\tau \dashv \varnothing} \ \texttt{(var)}$$

If x is declared in Γ, then x can be given the type τ *with all free variables replaced by* ***fresh variables***

This is where the polymorphism magic happens

HM⁻ (Typing Variables)

$$\frac{(x : \forall \alpha_1 . \forall \alpha_2 \ldots \forall \alpha_k . \tau) \in \Gamma \qquad \beta_1, \ldots, \beta_k \text{ are fresh}}{\Gamma \vdash x : [\beta_1/\alpha_1] \ldots [\beta_k/\alpha_k]\tau \dashv \varnothing} \text{ (var)}$$

If x is declared in Γ, then x can be given the type τ *with all free variables replaced by* ***fresh variables***

This is where the polymorphism magic happens

fresh variables can be unified with anything

HM^{-} (Typing Let-Expressions)

Let-Polymorphism

```
let f = fun x -> x in
let y = f 2 in
f true
```

Let-Polymorphism

```
let f = fun x -> x in
let y = f 2 in
f true
```

The "Problem": This rule doesn't allow let-defined functions to be polymorphic

Let-Polymorphism

```
let f = fun x -> x in
let y = f 2 in
f true
```

The "Problem": This rule doesn't allow let-defined functions to be polymorphic

(This is why we call our system Hindley-Milner *Light*)

Let-Polymorphism

```
let f = fun x -> x in
let y = f 2 in
f true
```

The "Problem": This rule doesn't allow let-defined functions to be polymorphic

(This is why we call our system Hindley-Milner *Light*)

There are some <u>debates</u> in the PL world on let-polymorphism...

Example

```
fun f -> fun x -> f (x + 1)
```

$\emptyset \vdash \text{fun } f \to \text{fun } x \to f\ (x+1)$

$\quad \vdash \{f : \alpha\} \vdash \text{fun } x \to f\ (x+1)$

$\quad\quad \vdash \{f : \alpha,\ x : \beta\} \vdash f\ (x+1)$

Up next: Unification

$$a \doteq d \to e$$
$$c \doteq \mathsf{int} \to d$$
$$\mathsf{int} \to \mathsf{int} \to \mathsf{int} \doteq b \to c$$

Up next: Unification

$$
\begin{aligned}
a &\doteq d \to e \\
c &\doteq \mathsf{int} \to d \\
\mathsf{int} \to \mathsf{int} \to \mathsf{int} &\doteq b \to c
\end{aligned}
$$

We need to introduce a **unification algorithm** for determining the "actual" type given a collection of constraints

Summary

By restricting our type quantification, we get a system that has decidable and efficient **type inference**

Hindley–Milner style type inference requires us to figure out a collection of **constraints** that need to be unified